Recursive algorithms (2/2)

Sapienza Università di Roma

Outline

Recursive Binary Search

The maximum subarray problem

Summary of the chapter

Where we were

- In the previous lesson we studied two recursive algorithms on arrays, with the scheme of the analysis of every recursive algorithm: pseudo-code and call tree, correctness by strong induction, recurrence (recursion tree and Master Theorem), running time, extra memory
- **The sum of an array**: `RecursiveSum` executes $7n - 5 = \Theta(n)$ steps, as the iterative algorithm, but uses $\Theta(\log_2 n)$ memory for the call stack ($\Theta(n)$ for the decrease-by-one version)
- **Merge Sort**: two recursive calls on the halves and a `Merge` in $\Theta(n)$ steps; $S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n)$, against $\Theta(n^2)$ of Insertion Sort

This lesson

- **Searching** in a sorted array, with the **Recursive Binary Search**: a single recursive call on one half
- The **maximum subarray** problem, where a divide and conquer algorithm improves on brute force, and a clever iterative algorithm improves on both
- At the end, a summary of the whole chapter

Recursive Binary Search

Introduction: Recursive Binary Search

- Merge Sort makes two recursive calls on the two halves. The next algorithm makes only **one**: after a single comparison, one of the two halves can be discarded
- We write the binary search of the chapter «Linear and binary search» as a recursive algorithm, and we analyse it with the same scheme
- Same running time as the iterative version, but more memory: this leads us to **tail recursion**

The problem

Given an array $a[1..n]$ of n natural numbers
sorted in non-decreasing order and a target natural number m,
we consider the problem of determining
if m is one of the elements of the array.

- In the chapter «Linear and binary search» we solved this problem with the **iterative** algorithm `BinarySearch`

The iterative algorithm

- The algorithm maintains a search interval $a[left..right]$, initially $a[1..n]$
- At each iteration, it compares the target m with the middle element

$$a[mid], \qquad mid = left + \left\lfloor \frac{right - left}{2} \right\rfloor .$$

- If $a[mid] = m$ the element is found; otherwise, since the array is sorted, half of the interval can be discarded

The same problem on a smaller sub-array

- The key observation is that, after the comparison with $a[mid]$, the problem that remains to be solved is **the same problem** (search m in a sorted array) on a **smaller** sub-array:
  - if $a[mid] < m$, then m can only be in $a[mid + 1..right]$;
  - if $a[mid] > m$, then m can only be in $a[left..mid - 1]$.

- This is exactly the structure required by a recursive algorithm: we solve the problem on $a[left..right]$ by solving the same problem on a sub-array of (about) half the size

Base cases and parameters

- The **base cases** are
  - the empty interval ($left > right$), in which m is certainly not present
  - the case $a[mid] = m$, in which m is found

- Differently from the iterative version, the boundaries *left* and *right* are now **parameters** of the function

- To search m in the whole array, we call

$$\texttt{RecursiveBinarySearch}(a, 1, n, m).$$

- In the next slides, the pseudo-code of the Recursive Binary Search algorithm

Algorithm: RecursiveBinarySearch (1/2)

Input : An array $a[1..n]$ of n natural numbers sorted in non-decreasing order, two indices *left* and *right*, and a target value m

Output : An index $i \in \{left, \ldots, right\}$ such that $a[i] = m$, or -1 if m is not in $a[left..right]$

```
1 function RecursiveBinarySearch(a, left, right, m)
2     if left > right then
3         return −1          // Empty interval: not found
4     end
5     mid := left + ⌊(right − left)/2⌋          // Middle index
6     if a[mid] = m then
7         return mid                          // Element found
8     end
```

Algorithm: RecursiveBinarySearch (2/2)

function *RecursiveBinarySearch(a, left, right, m)* (continued)

```
9     if a[mid] < m then
10        return RecursiveBinarySearch(a, mid + 1, right, m)
11    end
12    return RecursiveBinarySearch(a, left, mid − 1, m)
13 end
```

Run in Colab: Python · C++ · Java

How `RecursiveBinarySearch` works (1/2)

- Lines 2–3: if the interval is empty ($left > right$), the element is not present and -1 is returned (**base case**)
- Line 5: the middle index mid is computed with the same formula as in the iterative algorithm
- Lines 6–7: if $a[mid] = m$, the element is found and its index is returned (**base case**)

How `RecursiveBinarySearch` works (2/2)

- Lines 9–10: if $a[mid] < m$, the search continues **recursively** in the right half $a[mid + 1..right]$
- Line 12: otherwise $a[mid] > m$, and the search continues **recursively** in the left half $a[left..mid - 1]$
- The value returned by the recursive call is returned unchanged to the caller

Each call of `RecursiveBinarySearch` performs **at most one** recursive call.
Therefore, the **call tree** is a **path**: it has no branching.

This is very different from Merge Sort, where every call on an array with at least two elements generates two recursive calls.

Examples

- We consider the same searches performed in the chapter «Linear and binary search» with the iterative algorithm: we search four values in the sorted array

$$a = [3, 9, 11, 12, 12, 16, 19] \qquad (n = 7)$$

- For each search we report one row for each call: the values of *left*, *right*, *mid*, $a[mid]$ and the action executed

- Then, we draw the call tree: a black arrow is a recursive call, a blue arrow is the value returned to the caller

Example (1/6)

Search for $m = 12$. First call
`RecursiveBinarySearch`$(a, 1, 7, 12)$:

call	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] = m \Rightarrow$ Found! Return 4

`RecursiveBinarySearch`$(a, 1, 7, 12)$ $\longrightarrow$ returns 4

Element 12 is found at index 4 with a single call (no recursion).

Example (2/6)

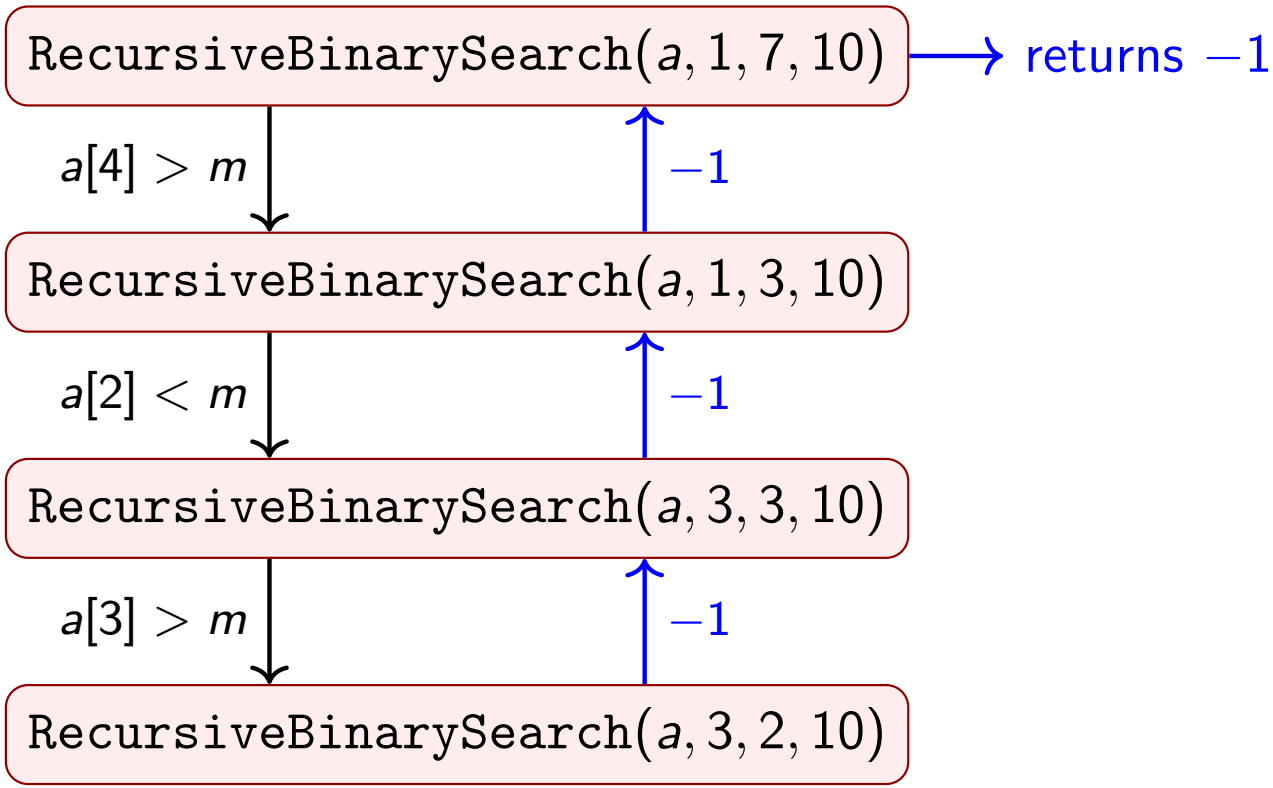
Search for $m = 10$. First call
`RecursiveBinarySearch`$(a, 1, 7, 10)$:

call	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] > m \Rightarrow$ call on $left = 1$, $right = 3$
2	1	3	2	9	$a[mid] < m \Rightarrow$ call on $left = 3$, $right = 3$
3	3	3	3	11	$a[mid] > m \Rightarrow$ call on $left = 3$, $right = 2$
4	3	2	–	–	$left > right \Rightarrow$ Not found! Return -1

Element 10 is not in the array: the last call is on an empty interval and returns -1, and this value is propagated back to the first call (call tree in the next slide).

Example (3/6)

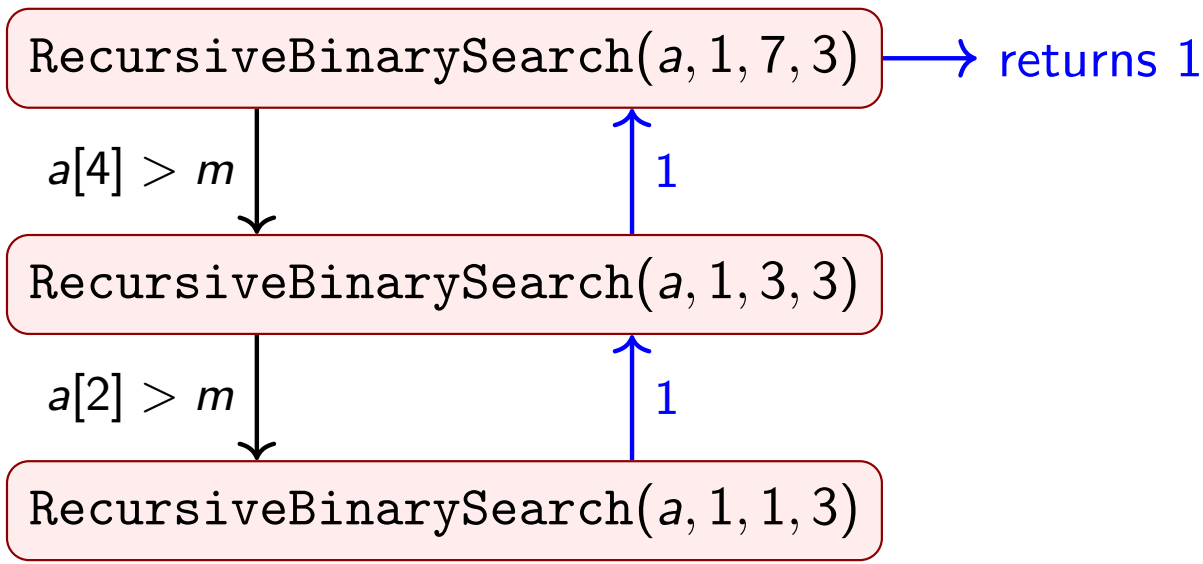
The call tree of the search for $m = 10$:



Example (4/6)

Search for $m = 3$. First call `RecursiveBinarySearch`$(a, 1, 7, 3)$:

call	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] > m \Rightarrow$ call on $left = 1$, $right = 3$
2	1	3	2	9	$a[mid] > m \Rightarrow$ call on $left = 1$, $right = 1$
3	1	1	1	3	$a[mid] = m \Rightarrow$ Found! Return 1



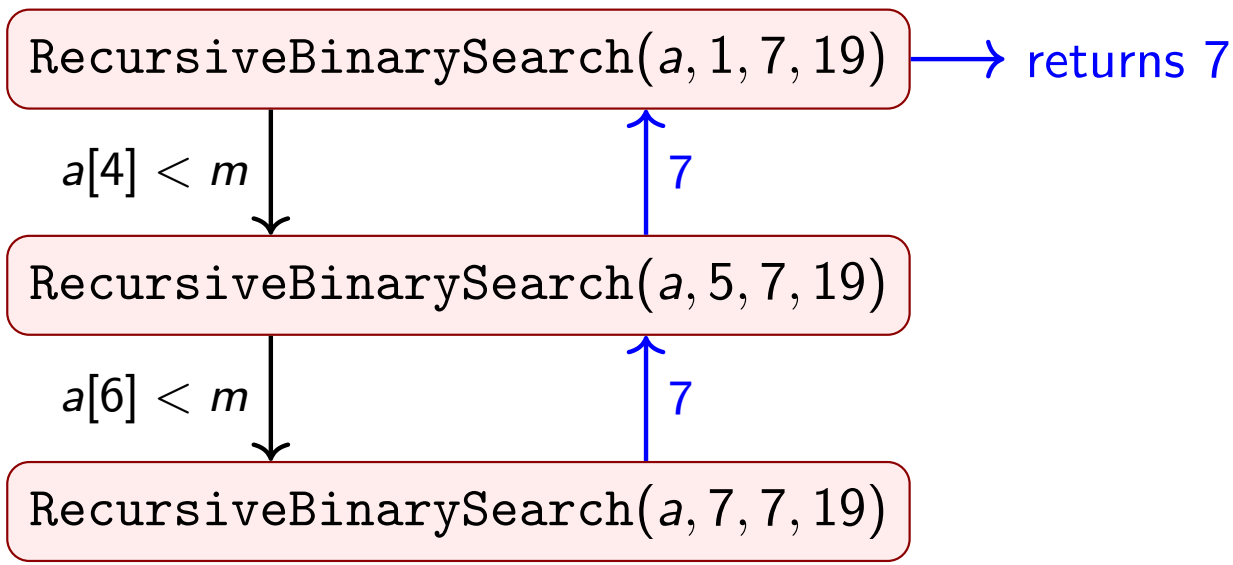
Element 3 is found at index 1.

Example (5/6)

Search for $m = 19$. First call

`RecursiveBinarySearch`$(a, 1, 7, 19)$:

call	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] < m \Rightarrow$ call on $left = 5$, $right = 7$
2	5	7	6	16	$a[mid] < m \Rightarrow$ call on $left = 7$, $right = 7$
3	7	7	7	19	$a[mid] = m \Rightarrow$ Found! Return 7



Element 19 is found at index 7.

Example (6/6)

- The sequence of values ($left$, $right$, mid) is **exactly the same** as the sequence produced by the iterative algorithm
- Each call of the recursive algorithm corresponds to one iteration of the `while` loop of the iterative algorithm
- Note that the second occurrence of 12 (index 5) is never found when searching $m = 12$
- As in the iterative version, the algorithm returns **one** index i with $a[i] = m$, not necessarily the first one

Correctness

- As for the previous algorithms, the correctness is proved by **strong induction** on the size of the input of the call
- The size of the call `RecursiveBinarySearch`$(a, \mathit{left}, \mathit{right}, m)$ is the number of elements of the interval:

$$k = \mathit{right} - \mathit{left} + 1.$$

- For an empty interval ($\mathit{left} = \mathit{right} + 1$) we have $k = 0$

Theorem 1: correctness of Recursive Binary Search

Let $a[1..n]$ *be sorted in non-decreasing order and let* $1 \leq left \leq right + 1 \leq n + 1$. *Then* `RecursiveBinarySearch`$(a, left, right, m)$ *terminates and returns:*

- *an index* $i \in \{left, \ldots, right\}$ *such that* $a[i] = m$, *if* m *is one of the elements of* $a[left..right]$;
- -1, *otherwise.*

Proof (1/3)

We proceed by strong induction on $k = \mathit{right} - \mathit{left} + 1 \geq 0$.

Base case $k = 0$. Then $\mathit{left} > \mathit{right}$, the interval is empty and m is not one of its elements. The algorithm executes lines 2–3 and returns -1, which is correct.

Inductive step. Let $k \geq 1$ and assume (inductive hypothesis) that the statement holds for every call whose interval has size $k' < k$. Since $\mathit{left} \leq \mathit{right}$, the algorithm computes

$$\mathit{mid} = \mathit{left} + \left\lfloor \frac{\mathit{right} - \mathit{left}}{2} \right\rfloor, \qquad \mathit{left} \leq \mathit{mid} \leq \mathit{right}.$$

Proof (2/3)

- If $a[mid] = m$, the algorithm returns $mid \in \{left, \ldots, right\}$, which is correct
- If $a[mid] < m$, then, since the array is sorted, $a[i] \leq a[mid] < m$ for every $i \in \{left, \ldots, mid\}$
  - Hence m is in $a[left..right]$ if and only if it is in $a[mid + 1..right]$
  - This interval has size $right - mid < k$ (because $mid \geq left$), so by the inductive hypothesis the call $\texttt{RecursiveBinarySearch}(a, mid + 1, right, m)$ terminates and returns a correct answer, which is returned unchanged

Proof (3/3)

- If $a[mid] > m$, then $a[i] \geq a[mid] > m$ for every $i \in \{mid, \ldots, right\}$
  - Hence m is in $a[left..right]$ if and only if it is in $a[left..mid-1]$, an interval of size $mid - left < k$ (because $mid \leq right$)
  - By the inductive hypothesis, the recursive call returns a correct answer

In all cases the call terminates and returns a correct answer. □

- Applying Theorem 1 with $left = 1$ and $right = n$, we obtain that `RecursiveBinarySearch`$(a, 1, n, m)$ solves the problem

Worst-case running time: the recurrence

- Let $S(n)$ denote the number of steps executed by `RecursiveBinarySearch` in the worst case on an interval of $n = right - left + 1$ elements (including the steps executed by all the recursive calls)

- The number of steps executed by a single call, **without** counting the steps of the recursive call, is:
  - empty interval: lines 2 and 3 $\Rightarrow$ 2 steps;
  - element found: lines 2, 5, 6, 7 $\Rightarrow$ 4 steps;
  - recursive call: lines 2, 5, 6, 9 and line 10 (or 12) $\Rightarrow$ 5 steps.

Size of the sub-problems

Let $n = right - left + 1 \geq 1$. Since $mid - left = \lfloor (n-1)/2 \rfloor$:

- the left interval $a[left..mid - 1]$ has

$$mid - left = \left\lfloor \frac{n-1}{2} \right\rfloor$$

  elements;
- the right interval $a[mid + 1..right]$ has

$$right - mid = (n-1) - \left\lfloor \frac{n-1}{2} \right\rfloor = \left\lceil \frac{n-1}{2} \right\rceil = \left\lfloor \frac{n}{2} \right\rfloor$$

  elements.

For example, for $n = 7$ both halves have 3 elements; for $n = 8$ the left half has 3 elements and the right half has 4.

The recurrence

- In the worst case the element is not found before the interval becomes empty, and the recursion continues on the larger half, which has $\lfloor n/2 \rfloor$ elements (one can easily check by induction that S is non-decreasing). Therefore,

$$S(n) = \begin{cases} 2 & \text{if } n = 0, \\ S\left(\lfloor n/2 \rfloor\right) + 5 & \text{if } n \geq 1. \end{cases}$$

- This worst case happens, for example, when $m > a[n]$: the algorithm always continues in the right half, until the interval becomes empty

- In general, we write $S(n) = S\left(\lfloor n/2 \rfloor\right) + c$, where c is a constant (here $c = 5$): a recursive algorithm that reduces the problem to **one** sub-problem of **half** the size, doing a **constant** amount of extra work

Solution by expansion (1/2)

We expand the recurrence, using the property $\lfloor \lfloor n/2^i \rfloor /2 \rfloor = \lfloor n/2^{i+1} \rfloor$:

$$\begin{aligned} S(n) &= S\big(\lfloor n/2 \rfloor\big) + 5 \\ &= S\big(\lfloor n/4 \rfloor\big) + 5 + 5 \\ &= S\big(\lfloor n/8 \rfloor\big) + 5 \cdot 3 \\ &\vdots \\ &= S\big(\lfloor n/2^j \rfloor\big) + 5\,j. \end{aligned}$$

The expansion stops at the first j such that $\lfloor n/2^j \rfloor = 0$, that is $2^j > n$. For $n \geq 1$, this is

$$j = \lfloor \log_2 n \rfloor + 1,$$

because $2^{\lfloor \log_2 n \rfloor} \leq n < 2^{\lfloor \log_2 n \rfloor + 1}$.

Solution by expansion (2/2)

- Hence, for every $n \geq 1$,

$$S(n) = 5\big(\lfloor \log_2 n \rfloor + 1\big) + 2.$$

- For example, $S(7) = 5 \cdot (2 + 1) + 2 = 17$: in the search for $m = 10$ of the examples, the three calls on non-empty intervals execute 5 steps each and the last call (empty interval) executes 2 steps

- Since $\log_2 n - 1 < \lfloor \log_2 n \rfloor \leq \log_2 n$, we have

$$\lim_{n \to \infty} \frac{S(n)}{\log_2 n} = \lim_{n \to \infty} \frac{5\lfloor \log_2 n \rfloor + 7}{\log_2 n} = 5 \in \mathbb{R}_{>0},$$

therefore

$$S(n) = \Theta(\log_2 n).$$

Solution by the Master Theorem

The same result is obtained immediately with the Master Theorem, applied to

$$S(n) = S(n/2) + c$$

(as usual, floors do not change the asymptotic solution).

- **Identify** a, b, $f(n)$: $a = 1$, $b = 2$, $f(n) = c$
- **Compute** $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$
- **Compare**: $f(n) = c = \Theta(1) = \Theta\left(n^{\log_2 1} \log^0 n\right)$
- **Verify the hypothesis**: we are in case 2 of the Master Theorem with $k = 0$
- **Conclude**:

$$S(n) = \Theta\left(n^{\log_2 1} \log^{0+1} n\right) = \Theta(\log n) = \Theta(\log_2 n).$$

Number of recursive calls

- Each call on a non-empty interval compares m with exactly one element $a[mid]$
- Therefore, the number of calls on a non-empty interval is the number of elements of the array examined by the algorithm (exactly as the number of iterations of the `while` loop of the iterative algorithm)

Observation 1: number of calls of Recursive Binary Search

For $n \geq 1$, the call `RecursiveBinarySearch`$(a, 1, n, m)$ *executes at most*

$$\lfloor \log_2 n \rfloor + 1$$

calls on a non-empty interval, and this bound is attained (for example when $m > a[n]$). Counting also the final call on an empty interval, the total number of calls is at most $\lfloor \log_2 n \rfloor + 2$.

Proof (1/3)

Let n_i be the size of the interval of the i-th call ($i = 1$ is the first call, so $n_1 = n$).

By the analysis of the sub-problems, $n_{i+1} \leq \lfloor n_i/2 \rfloor$. By induction on i,

$$n_i \leq \left\lfloor \frac{n}{2^{i-1}} \right\rfloor :$$

it holds for $i = 1$, and if it holds for i then

$$n_{i+1} \leq \lfloor n_i/2 \rfloor \leq \left\lfloor \lfloor n/2^{i-1} \rfloor /2 \right\rfloor = \lfloor n/2^i \rfloor .$$

Proof (2/3)

If the i-th call is on a non-empty interval, then $n_i \geq 1$, hence $n/2^{i-1} \geq 1$, i.e.,

$$2^{i-1} \leq n \qquad \text{and} \qquad i - 1 \leq \log_2 n.$$

Since $i - 1$ is an integer, $i - 1 \leq \lfloor \log_2 n \rfloor$, i.e.,

$$i \leq \lfloor \log_2 n \rfloor + 1.$$

Thus, at most $\lfloor \log_2 n \rfloor + 1$ calls are on a non-empty interval, and at most one further call (on an empty interval, which makes no recursive call) can follow.

Proof (3/3)

If $m > a[n]$, the algorithm always continues in the right half, whose size is exactly $\lfloor n_i/2 \rfloor$.

Hence

$$n_i = \lfloor n/2^{i-1} \rfloor,$$

which is ≥ 1 for $i = \lfloor \log_2 n \rfloor + 1$: the bound is attained, and it is followed by one call on an empty interval. □

- For example, for $n = 7$ we have $\lfloor \log_2 7 \rfloor + 1 = 3$: in the search of $m = 10$ there are 3 calls on non-empty intervals, plus the final call on the empty interval $left = 3$, $right = 2$
- The same reasoning shows that also the iterative algorithm executes at most $\lfloor \log_2 n \rfloor + 1$ iterations of the `while` loop

Running time as a function of the input size

- The input of the algorithm consists of the array $a[1..n]$ and of the target m
- Under the hypothesis that every value uses a constant number of bits, the input size satisfies $|I| = \Theta(n)$, that is, there exist constants $c_1, c_2 > 0$ such that $c_1 n \leq |I| \leq c_2 n$ for all sufficiently large n
- Taking logarithms,

$$\log_2 |I| - \log_2 c_2 \leq \log_2 n \leq \log_2 |I| - \log_2 c_1,$$

  hence $\log_2 n = \Theta(\log_2 |I|)$ (the additive constants are negligible with respect to $\log_2 |I|$)
- Since $S(n) = \Theta(\log_2 n)$, we obtain $S(n) = \Theta(\log_2 |I|)$

Worst-case running time of `RecursiveBinarySearch`

If all input values use a fixed constant number of bits,
then the worst-case running time
of the `RecursiveBinarySearch` algorithm is

$$T(|I|) = \Theta(\log |I|).$$

- As the iterative version, the algorithm is logarithmic in the input size
- It is **not** necessary to read the whole input to solve the problem, because the array is sorted

Space: recursive versus iterative Binary Search (1/2)

- The recursive and the iterative algorithms execute the same comparisons and have the same worst-case running time. They differ in the **memory** used
- Every call that has not yet terminated keeps in memory its own copy of the parameters and local variables (*left*, *right*, *m*, *mid*, a reference to the array, and the return address)
- This information is stored in the **call stack**, and it occupies a constant amount of memory per call

Space: recursive versus iterative Binary Search (2/2)

- In `RecursiveBinarySearch` no call terminates before its recursive call has terminated
- Therefore, when the last call is executing, **all** the calls of the path are in the call stack
- By Observation 1, in the worst case there are $\lfloor \log_2 n \rfloor + 2$ of them, and the extra memory used is

$$\Theta(\log n).$$

- The iterative algorithm uses only the variables *left*, *right* and *mid*: its extra memory is

$$\Theta(1).$$

Tail recursion

Definition 1: tail recursion

*A recursive call is a **tail call** if it is the last operation executed by the calling function, i.e., the value it returns is directly returned by the caller without any further computation. A function whose recursive calls are all tail calls is called **tail recursive**.*

- `RecursiveBinarySearch` is tail recursive: in lines 10 and 12 the value of the recursive call is returned as it is

From recursion to iteration

- A tail call can be replaced by updating the parameters and jumping back to the beginning of the function:
  - line 10 becomes $left \leftarrow mid + 1$;
  - line 12 becomes $right \leftarrow mid - 1$;
  - the test of line 2 becomes the condition of a `while` loop "**while** $left \leq right$".

- The result of this transformation is **exactly** the iterative `BinarySearch` algorithm

- This is why the two algorithms produce the same sequence of values of *left*, *right* and *mid*

Tail-call elimination

- Many compilers perform this transformation automatically (**tail-call elimination**), so that the recursive version uses only $\Theta(1)$ extra memory
- For instance, the compiler g++ 15.2 removes the recursive calls of the C++ implementation with the option `-O2`, but not with `-O0`
- However, this is not guaranteed by the C++ standard

Recursive versus iterative Binary Search

Recursive and iterative Binary Search
have the same worst-case running time $\Theta(\log n)$,
but the recursive version uses $\Theta(\log n)$ extra memory
for the call stack, while the iterative version uses
$\Theta(1)$ extra memory.

Every tail-recursive algorithm
can be transformed into an iterative one.

Summary

Key points on the Recursive Binary Search:

- One recursive call on (at most) half of the interval: the call tree is a path, with at most $\lfloor \log_2 n \rfloor + 2$ calls
- $S(n) = S(\lfloor n/2 \rfloor) + 5$, i.e., $S(n) = 5(\lfloor \log_2 n \rfloor + 1) + 2 = \Theta(\log n)$ (case 2 of the Master Theorem with $a = 1$)
- The algorithm is tail recursive: removing the recursion gives the iterative `BinarySearch`, with $\Theta(1)$ extra memory instead of $\Theta(\log n)$

The maximum subarray problem

Introduction: The maximum subarray problem

- The last problem of the chapter shows divide and conquer at work on a less obvious problem, where the *combine* step is the interesting part
- Three algorithms: brute force ($\Theta(n^2)$), divide and conquer ($\Theta(n \log n)$), and an iterative algorithm ($\Theta(n)$)
- A recursive algorithm that improves on brute force is not necessarily the best possible one

Motivation: buying and selling a stock

- Suppose we know in advance the price of a stock for the next days
- We may buy one unit of stock **once** and sell it **once**, at a later day. We want to maximise the profit (sell price minus buy price)
- In the example of the next slide, $p[d]$ is the price at day d, and

$$a[d] = p[d] - p[d-1]$$

  is the **daily change** of the price

The example is taken from the textbook of T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, Section 4.1.

Prices and daily changes

day d	0	1	2	3	4	5	6	7	8
price $p[d]$	100	113	110	85	105	102	86	63	81
change $a[d]$		13	-3	-25	20	-3	-16	-23	18

day d	9	10	11	12	13	14	15	16
price $p[d]$	101	94	106	101	79	94	90	97
change $a[d]$	20	-7	12	-5	-22	15	-4	7

The profit is a sum of daily changes

- If we buy at day $l-1$ and sell at day $r \geq l$, the profit is

$$\begin{aligned} p[r] - p[l-1] &= \big(p[l] - p[l-1]\big) + \big(p[l+1] - p[l]\big) \\ &\quad + \cdots + \big(p[r] - p[r-1]\big) \\ &= a[l] + a[l+1] + \cdots + a[r], \end{aligned}$$

  i.e., the sum of the consecutive daily changes $a[l..r]$

- Hence, maximising the profit is equivalent to finding the consecutive portion of the array $a[1..16]$ with the **largest sum**

Buying low and selling high is not enough

- Note that it is **not** enough to buy at the lowest price or to sell at the highest price: the lowest price (63, day 7) comes after the highest price (113, day 1)

- In this example the best choice is to buy at day 7 (price 63) and sell at day 11 (price 106), with profit

$$106 - 63 = 43 = a[8] + a[9] + a[10] + a[11] = 18 + 20 - 7 + 12.$$

The problem

Given an array $a[1..n]$ of n integer numbers,
consider the problem of finding two indices $l \leq r$ such that the sum

$$a[l] + a[l+1] + \cdots + a[r] = \sum_{k=l}^{r} a[k]$$

is maximum.

The sub-array $a[l..r]$ is called a **maximum subarray** of a.

Remarks on the problem

- A subarray is always **non-empty** ($l \leq r$) and made of **consecutive** elements
- The problem is interesting only if the array contains negative numbers:
  - if all the entries are non-negative, the whole array $a[1..n]$ is a maximum subarray
  - if all the entries are negative, a maximum subarray is made of a single element, the largest one
- A maximum subarray is not necessarily unique: we look for **one** of them
- The number of subarrays of $a[1..n]$ is the number of pairs (l, r) with $1 \leq l \leq r \leq n$:

$$\sum_{l=1}^{n}(n-l+1) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2}.$$

A brute force algorithm

- The simplest algorithm considers all the $n(n+1)/2$ subarrays $a[l..r]$ and keeps the best one
- For a fixed l, the sums of $a[l..r]$ for $r = l, l+1, \ldots, n$ are computed incrementally: the sum of $a[l..r]$ is the sum of $a[l..r-1]$ plus $a[r]$
- In the next slides, the pseudo-code of the `BruteForceMaximumSubarray` algorithm

Algorithm: BruteForceMaximumSubarray (1/2)

Input : An array $a[1..n]$ of n integer numbers
Output : Indices $bl \leq br$ of a maximum subarray and its sum *best*

```
1  function BruteForceMaximumSubarray(a, n)
2      best := a[1]
3      bl := 1
4      br := 1
5      l := 1
6      while l ≤ n do
7          sum := 0
8          r := l
```

Algorithm: BruteForceMaximumSubarray (2/2)

```
function BruteForceMaximumSubarray(a, n) (continued)
    while l ≤ n (continued) do
 9      while r ≤ n do
10          sum ← sum + a[r]                    // sum of a[l..r]
11          if sum > best then
12              best ← sum
13              bl ← l
14              br ← r
15          end
16          r ← r + 1
17      end
18      l ← l + 1
19  end
20  return (bl, br, best)
21 end
```

Run in Colab: Python · C++ · Java

Example

Consider $a = [-2, 5, -1, 3]$. The algorithm starts with $best = a[1] = -2$ and examines the $4 \cdot 5/2 = 10$ subarrays in this order:

l	r	sum	action
1	1	-2	$sum \leq best = -2$
1	2	3	$sum > best \Rightarrow best = 3,\ bl = 1,\ br = 2$
1	3	2	
1	4	5	$sum > best \Rightarrow best = 5,\ bl = 1,\ br = 4$
2	2	5	$sum \leq best = 5$
2	3	4	
2	4	7	$sum > best \Rightarrow best = 7,\ bl = 2,\ br = 4$
3	3	-1	
3	4	2	
4	4	3	

The algorithm returns $(2, 4, 7)$: the maximum subarray is $a[2..4] = [5, -1, 3]$ with sum 7.

Worst-case running time (1/4)

- Let $S(n)$ denote the number of steps executed by `BruteForceMaximumSubarray` in the worst case, where n is the size of the array

- The execution of lines 12–14 depends on the input. We define a function $t(l, r) \in \{0, 1\}$, for $1 \leq l \leq r \leq n$, where $t(l, r) = 1$ if lines 12–14 are executed when the inner loop examines the pair (l, r), and $t(l, r) = 0$ otherwise

- For a fixed l, the inner loop executes $n - l + 1$ iterations, and its condition is tested $n - l + 2$ times. Hence

$$\sum_{l=1}^{n}(n-l+1) = \frac{n(n+1)}{2}, \qquad \sum_{l=1}^{n}(n-l+2) = \frac{n(n+1)}{2} + n.$$

Worst-case running time (2/4)

The number of times each line of code is executed is:

lines	number of executions (each)
2, 3, 4, 5	1
6	$n+1$
7, 8	n
9	$\frac{n(n+1)}{2}+n$
10, 11, 16	$\frac{n(n+1)}{2}$
12, 13, 14	$\sum_{l=1}^{n}\sum_{r=l}^{n} t(l,r)$
18	n
20	1

Worst-case running time (3/4)

- Therefore, the total number of steps is

$$4 + (n+1) + 2n + \frac{n(n+1)}{2} + n + 3\,\frac{n(n+1)}{2} + n + 1$$
$$+ 3\sum_{l=1}^{n}\sum_{r=l}^{n} t(l,r)$$
$$= 2n^2 + 7n + 6 + 3\sum_{l=1}^{n}\sum_{r=l}^{n} t(l,r).$$

- In the example there are 3 updates and the algorithm executes

$$2 \cdot 16 + 7 \cdot 4 + 6 + 3 \cdot 3 = 75$$

steps

Worst-case running time (4/4)

- Since $0 \leq t(l, r) \leq 1$, the double sum is between 0 and $n(n+1)/2$. Hence, for every input of size n, the number of steps is between

$$2n^2+7n+6 \quad \text{and} \quad 2n^2+7n+6+\frac{3n(n+1)}{2} = \frac{7}{2}n^2+\frac{17}{2}n+6,$$

  and so is the worst-case number of steps $S(n)$
- Both bounds are $\Theta(n^2)$; for example,

$$\lim_{n\to\infty} \frac{2n^2+7n+6}{n^2} = 2 \in \mathbb{R}_{>0},$$

$$\lim_{n\to\infty} \frac{\frac{7}{2}n^2+\frac{17}{2}n+6}{n^2} = \frac{7}{2} \in \mathbb{R}_{>0}.$$

  Therefore, $S(n) = \Theta(n^2)$

The brute force algorithm is quadratic

- Note that, for **every** input (not only in the worst case), the algorithm executes $\Theta(n^2)$ steps, since it always examines all the $n(n+1)/2$ subarrays
- Under the hypothesis that every entry uses a constant number of bits, $|I| = \Theta(n)$, and therefore $S(n) = \Theta(|I|^2)$

If all input values use a fixed constant number of bits,
then the worst-case running time
of the `BruteForceMaximumSubarray` algorithm is

$$T(|I|) = \Theta(|I|^2).$$

Thus, the brute force algorithm is quadratic in the input size.

A divide and conquer algorithm (1/2)

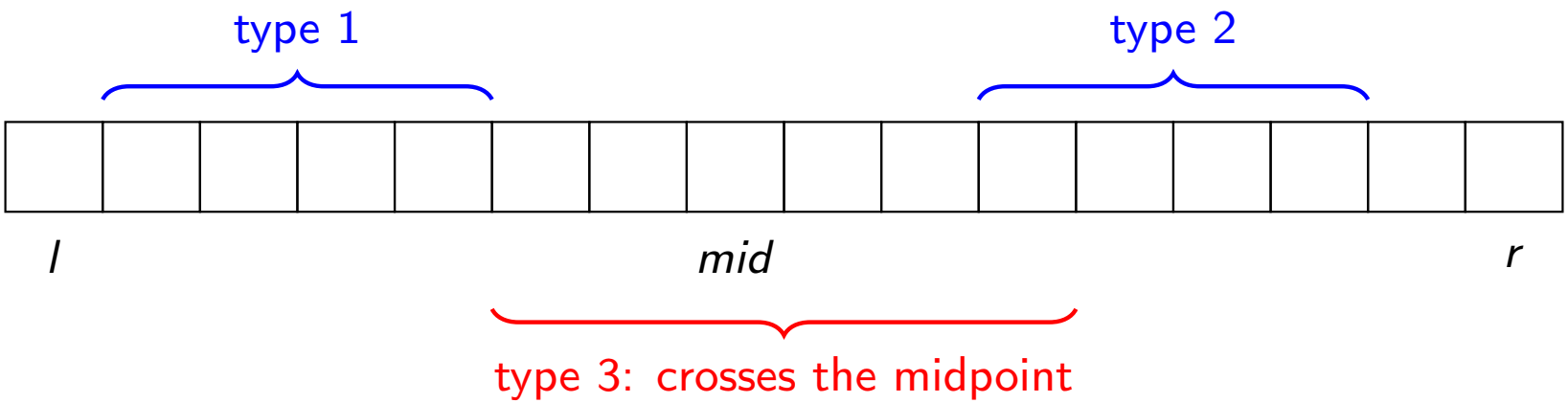
- We now design a recursive algorithm based on the **divide and conquer** approach, as for Merge Sort
- We solve the problem on a sub-array $a[l..r]$, and we initially call it with $l = 1$ and $r = n$
- **Divide**: split $a[l..r]$ into two halves $a[l..mid]$ and $a[mid + 1..r]$, where

$$mid = \left\lfloor \frac{l + r}{2} \right\rfloor \quad \left(= l + \left\lfloor \frac{r - l}{2} \right\rfloor \right).$$

- Every subarray $a[i..j]$ of $a[l..r]$ (with $l \leq i \leq j \leq r$) is in exactly one of these three situations:
  1. it lies entirely in the left half: $l \leq i \leq j \leq mid$;
  2. it lies entirely in the right half: $mid + 1 \leq i \leq j \leq r$;
  3. it **crosses the midpoint**: $l \leq i \leq mid < j \leq r$.

A divide and conquer algorithm (2/2)

- **Conquer**: a maximum subarray of type 1 and of type 2 are found **recursively**, since they are maximum subarrays of $a[l..mid]$ and $a[mid + 1..r]$, respectively
- **Combine**: a maximum subarray of type 3 is found by a non-recursive procedure, `FindMaxCrossingSubarray`, described below. The answer is the best of the three subarrays



Maximum subarray crossing the midpoint

- A subarray crossing the midpoint is made of two parts: $a[i..mid]$, which **ends** at mid, and $a[mid+1..j]$, which **starts** at $mid+1$:

$$\sum_{k=i}^{j} a[k] = \sum_{k=i}^{mid} a[k] + \sum_{k=mid+1}^{j} a[k].$$

- The first term depends only on i, the second only on j: the two parts can be chosen **independently**
- Therefore, it is enough to find the best i (scanning the left half from mid down to l) and the best j (scanning the right half from $mid+1$ up to r)
- In the next slides, the pseudo-code of the `FindMaxCrossingSubarray` algorithm (from the same textbook as the stock example)

Algorithm: FindMaxCrossingSubarray (1/2)

Input : An array a and three indices $l \leq mid < r$
Output : Indices $maxleft \leq mid < maxright$ of a maximum subarray crossing the midpoint, and its sum

```
1  function FindMaxCrossingSubarray(a, l, mid, r)
2      leftsum := −∞
3      sum := 0
4      for i := mid downto l do
5          sum ← sum + a[i]                    // sum of a[i..mid]
6          if sum > leftsum then
7              leftsum ← sum
8              maxleft ← i
9          end
10     end
```

Algorithm: FindMaxCrossingSubarray (2/2)

function *FindMaxCrossingSubarray(a, l, mid, r)* (continued)

```
11      rightsum := −∞
12      sum ← 0
13      for j := mid + 1 to r do
14          sum ← sum + a[j]                    // sum of a[mid + 1..j]
15          if sum > rightsum then
16              rightsum ← sum
17              maxright ← j
18          end
19      end
20      return (maxleft, maxright, leftsum + rightsum)
21  end
```

Run in Colab: Python · C++ · Java

Lemma 1: crossing subarray

Let $l \leq mid < r$. `FindMaxCrossingSubarray`(a, l, mid, r) *returns* $(maxleft, maxright, s)$ *with* $l \leq maxleft \leq mid < maxright \leq r$ *and*

$$s = \sum_{k=maxleft}^{maxright} a[k]$$

$$= \max \left\{ \sum_{k=i}^{j} a[k] \ : \ l \leq i \leq mid < j \leq r \right\}.$$

Moreover, it executes $\Theta(n)$ *steps, where* $n = r - l + 1$.

Proof (1/3)

Left loop. At the end of the iteration with index i:

- the variable *sum* is equal to

$$L(i) = \sum_{k=i}^{mid} a[k];$$

- *leftsum* is the maximum of $L(mid), L(mid-1), \ldots, L(i)$, attained at *maxleft*

(by induction on the iterations: at each iteration *sum* is compared with the best value found so far, which is initially $-\infty$, so the first comparison always succeeds).

At the end of the loop,

$$leftsum = \max_{l \leq i \leq mid} L(i) = L(maxleft).$$

Proof (2/3)

Right loop. In the same way, with $R(j) = \sum_{k=mid+1}^{j} a[k]$,

$$rightsum = \max_{mid+1 \le j \le r} R(j) = R(maxright).$$

Combination. For every crossing subarray, $\sum_{k=i}^{j} a[k] = L(i) + R(j)$. Since i and j can be chosen independently,

$$\begin{aligned} \max_{l \le i \le mid < j \le r} \big(L(i) + R(j)\big) &= \max_{l \le i \le mid} L(i) + \max_{mid < j \le r} R(j) \\ &= leftsum + rightsum, \end{aligned}$$

which is the value returned, attained by $a[maxleft..maxright]$.

Proof (3/3)

Running time.

- The two loops execute $(mid - l + 1) + (r - mid) = r - l + 1 = n$ iterations in total
- Each iteration executes a constant number of steps (between 3 and 5 lines, counting the loop test)
- In addition, a constant number of steps outside the loops

Hence the number of steps is between $c_1 n$ and $c_2 n$ for two constants $c_1, c_2 > 0$, i.e., $\Theta(n)$. □

- Note that the crossing subarray always contains both $a[mid]$ and $a[mid + 1]$, even if they are negative: the subarrays of type 1 and 2 take care of the other cases

The recursive algorithm

In the next slides, the pseudo-code
of the `FindMaximumSubarray` algorithm.

To solve the problem on the whole array
we call `FindMaximumSubarray`$(a, 1, n)$.

Algorithm: FindMaximumSubarray (1/2)

Input : An array a and two indices $l \leq r$
Output : Indices $i \leq j$ of a maximum subarray of $a[l..r]$ and its sum

```
1  function FindMaximumSubarray(a, l, r)
2      if l = r then
3          return (l, r, a[l])      // Base case: one element
4      end
5      mid := ⌊(l + r)/2⌋
6      (ll, lr, lsum) := FindMaximumSubarray(a, l, mid)
7      (rl, rr, rsum) := FindMaximumSubarray(a, mid + 1, r)
8      (cl, cr, csum) := FindMaxCrossingSubarray(a, l, mid, r)
```

Algorithm: FindMaximumSubarray (2/2)

function *FindMaximumSubarray(a, l, r)* (continued)

```
9     if lsum ≥ rsum and lsum ≥ csum then
10        return (ll, lr, lsum)
11    end
12    if rsum ≥ lsum and rsum ≥ csum then
13        return (rl, rr, rsum)
14    end
15    return (cl, cr, csum)
16 end
```

Run in Colab: Python · C++ · Java

How `FindMaximumSubarray` works

- Lines 2–3: if the sub-array has only one element, its only subarray is $a[l..l]$ (**base case**)
- Line 5: **divide** the sub-array into $a[l..mid]$ and $a[mid + 1..r]$
- Lines 6–7: **conquer**, i.e., find recursively a maximum subarray in each half
- Line 8: find a maximum subarray crossing the midpoint
- Lines 9–15: **combine**, i.e., return the best of the three (in case of ties, the left one is preferred, then the right one)

- Differently from the Recursive Binary Search, each call on at least two elements makes **two** recursive calls: the call tree is a binary tree with n leaves (one for each element) and $n - 1$ internal nodes, i.e., $2n - 1$ calls in total, exactly as for `RecursiveSum`

Example: the stock example

Consider the array of the daily changes of the stock example:

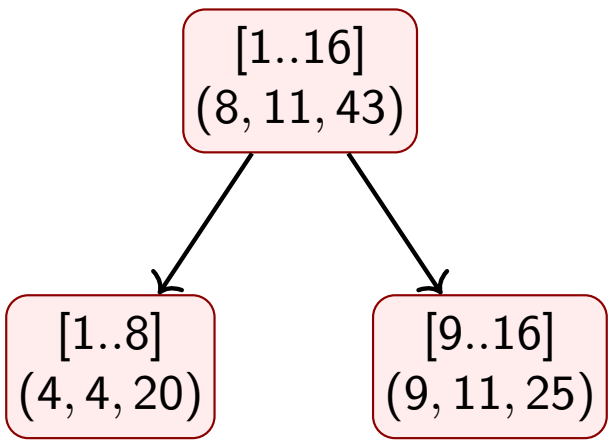
i	1	2	3	4	5	6	7	8
$a[i]$	13	−3	−25	20	−3	−16	−23	18

i	9	10	11	12	13	14	15	16
$a[i]$	20	−7	12	−5	−22	15	−4	7

We run the divide and conquer algorithm in four steps: the call tree, the results of the calls, the crossing computations of the two calls of level 1, the crossing computation of the first call.

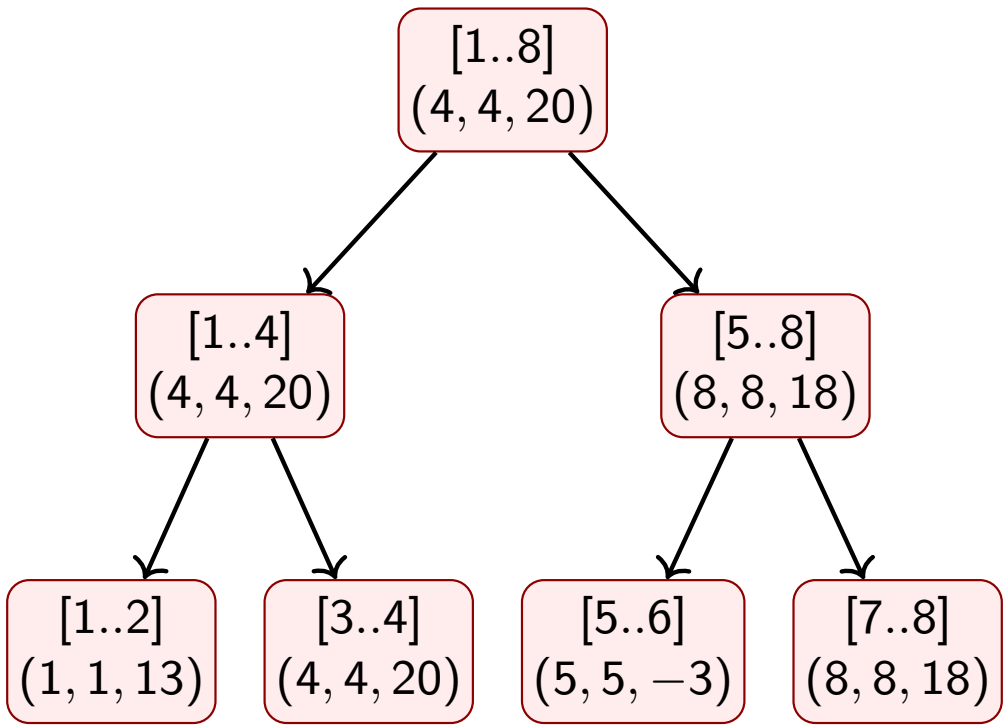
Example: step 1, the call tree (1/3)

- We call `FindMaximumSubarray`$(a, 1, 16)$
- Each node is a call on $a[l..r]$, written as $[l..r]$, together with the triple (i, j, s) it returns
- The 16 leaves (calls on one element $[k..k]$, returning $(k, k, a[k])$) are not drawn
- First the root and its two children, then the sub-trees of [1..8] and of [9..16]



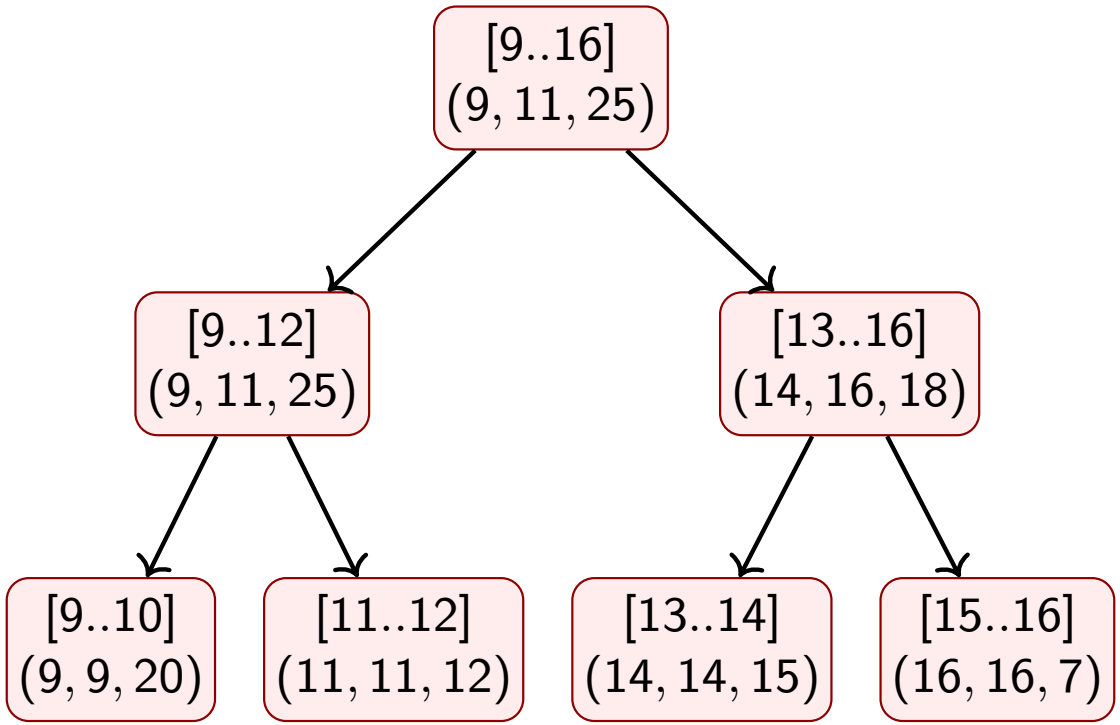
Example: step 1, the call tree (2/3)

The sub-tree of the call [1..8]:



Example: step 1, the call tree (3/3)

The sub-tree of the call [9..16]:



Example: step 2, the results of the calls (1/2)

For each call on at least two elements: the triples returned by the left recursive call, by the right recursive call and by `FindMaxCrossingSubarray`, and the triple returned (the best of the three).

call	*mid*	left	right	crossing	returned
[1..16]	8	(4, 4, 20)	(9, 11, 25)	(8, 11, 43)	(8, 11, 43)
[1..8]	4	(4, 4, 20)	(8, 8, 18)	(4, 5, 17)	(4, 4, 20)
[9..16]	12	(9, 11, 25)	(14, 16, 18)	(9, 16, 16)	(9, 11, 25)
[1..4]	2	(1, 1, 13)	(4, 4, 20)	(1, 4, 5)	(4, 4, 20)
[5..8]	6	(5, 5, −3)	(8, 8, 18)	(6, 8, −21)	(8, 8, 18)
[9..12]	10	(9, 9, 20)	(11, 11, 12)	(9, 11, 25)	(9, 11, 25)
[13..16]	14	(14, 14, 15)	(16, 16, 7)	(14, 16, 18)	(14, 16, 18)

Example: step 2, the results of the calls (2/2)

The calls on two elements:

call	*mid*	left	right	crossing	returned
[1..2]	1	$(1, 1, 13)$	$(2, 2, -3)$	$(1, 2, 10)$	$(1, 1, 13)$
[3..4]	3	$(3, 3, -25)$	$(4, 4, 20)$	$(3, 4, -5)$	$(4, 4, 20)$
[5..6]	5	$(5, 5, -3)$	$(6, 6, -16)$	$(5, 6, -19)$	$(5, 5, -3)$
[7..8]	7	$(7, 7, -23)$	$(8, 8, 18)$	$(7, 8, -5)$	$(8, 8, 18)$
[9..10]	9	$(9, 9, 20)$	$(10, 10, -7)$	$(9, 10, 13)$	$(9, 9, 20)$
[11..12]	11	$(11, 11, 12)$	$(12, 12, -5)$	$(11, 12, 7)$	$(11, 11, 12)$
[13..14]	13	$(13, 13, -22)$	$(14, 14, 15)$	$(13, 14, -7)$	$(14, 14, 15)$
[15..16]	15	$(15, 15, -4)$	$(16, 16, 7)$	$(15, 16, 3)$	$(16, 16, 7)$

Example: step 3, the crossing computations of level 1 (1/2)

Call [1..8], $mid = 4$:

i	$a[i]$	sum	$leftsum$	$maxleft$
4	20	20	20	4
3	−25	−5	20	4
2	−3	−8	20	4
1	13	5	20	4

j	$a[j]$	sum	$rightsum$	$maxright$
5	−3	−3	−3	5
6	−16	−19	−3	5
7	−23	−42	−3	5
8	18	−24	−3	5

The crossing subarray is $a[4..5]$ with sum $20 + (-3) = 17$.

Example: step 3, the crossing computations of level 1 (2/2)

Call [9..16], $mid = 12$:

i	$a[i]$	sum	$leftsum$	$maxleft$
12	-5	-5	-5	12
11	12	7	7	11
10	-7	0	7	11
9	20	20	20	9

j	$a[j]$	sum	$rightsum$	$maxright$
13	-22	-22	-22	13
14	15	-7	-7	14
15	-4	-11	-7	14
16	7	-4	-4	16

The crossing subarray is $a[9..16]$ with sum $20 + (-4) = 16$.

Example: step 4, the crossing computation of the first call (1/3)

Call [1..16], $mid = 8$, left loop:

i	$a[i]$	sum	$leftsum$	$maxleft$
8	18	18	18	8
7	−23	−5	18	8
6	−16	−21	18	8
5	−3	−24	18	8
4	20	−4	18	8
3	−25	−29	18	8
2	−3	−32	18	8
1	13	−19	18	8

Example: step 4, the crossing computation of the first call (2/3)

Call [1..16], $mid = 8$, right loop:

j	$a[j]$	sum	$rightsum$	$maxright$
9	20	20	20	9
10	-7	13	20	9
11	12	25	25	11
12	-5	20	25	11
13	-22	-2	25	11
14	15	13	25	11
15	-4	9	25	11
16	7	16	25	11

The crossing subarray is $a[8..11]$ with sum $18 + 25 = 43$.

Example: step 4, the crossing computation of the first call (3/3)

- In the first call, the left half gives 20, the right half gives 25 and the crossing subarray gives 43
- The algorithm returns $(8, 11, 43)$, i.e., buy at day 7 and sell at day 11, with profit 43
- Note that the optimal subarray is found only at the last step, by `FindMaxCrossingSubarray`: it is neither in the left half nor in the right half

Theorem 2: correctness of FindMaximumSubarray

For every $l \leq r$, `FindMaximumSubarray`(a, l, r) *returns* (i, j, s) *with* $l \leq i \leq j \leq r$ *and*

$$s = \sum_{k=i}^{j} a[k] = \max \left\{ \sum_{k=i'}^{j'} a[k] \ : \ l \leq i' \leq j' \leq r \right\}.$$

- As for all the recursive algorithms of the chapter, the proof is by strong induction on the number of elements of the sub-array

Proof (1/2)

We proceed by strong induction on the number of elements $n = r - l + 1 \geq 1$.

Base case $n = 1$. Then $l = r$, the only subarray is $a[l..l]$, and the algorithm returns $(l, l, a[l])$, which is correct.

Inductive step. Let $n \geq 2$ and assume (inductive hypothesis) that the statement holds for every sub-array with $n' < n$ elements. Since $l < r$, we have $l \leq mid < r$, and the two halves $a[l..mid]$ and $a[mid + 1..r]$ have $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$ elements, both at least 1 and less than n.

Proof (2/2)

By the inductive hypothesis:

- *lsum* is the maximum sum of a subarray of type 1 (contained in $a[l..mid]$), attained by $a[ll..lr]$;
- *rsum* is the maximum sum of a subarray of type 2 (contained in $a[mid+1..r]$), attained by $a[rl..rr]$.

By Lemma 1, *csum* is the maximum sum of a subarray of type 3 (crossing the midpoint), attained by $a[cl..cr]$.

Every subarray of $a[l..r]$ is of type 1, 2 or 3. Hence the maximum sum of a subarray of $a[l..r]$ is

$$\max\{lsum, rsum, csum\},$$

and lines 9–15 return exactly a triple attaining this maximum. □

Worst-case running time: the recurrence

- Let $S(n)$ denote the number of steps executed by `FindMaximumSubarray` in the worst case on a sub-array of $n = r - l + 1$ elements
- If $n = 1$, only lines 2–3 are executed: $S(1) = c_1$, a constant
- If $n \geq 2$:
  - lines 2, 5 and 9–15 execute a constant number of steps;
  - the recursive calls of lines 6 and 7 execute $S(\lceil n/2 \rceil)$ and $S(\lfloor n/2 \rfloor)$ steps;
  - line 8 executes $\Theta(n)$ steps (Lemma 1).
- Therefore,

$$S(n) = \begin{cases} c_1 & \text{if } n = 1, \\ S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + \Theta(n) & \text{if } n \geq 2. \end{cases}$$

The same recurrence as Merge Sort

- Ignoring floors and ceilings (which do not change the asymptotic solution), we obtain the same recurrence as Merge Sort:

$$S(n) = 2\,S(n/2) + \Theta(n).$$

- Note that the running time does not depend on the values in the array, but only on n: every input of size n is a worst case

Solution by the Master Theorem:

- **Identify** a, b, $f(n)$: $a = 2$, $b = 2$, $f(n) = \Theta(n)$
- **Compute** $n^{\log_b a} = n^{\log_2 2} = n$
- **Compare**: $f(n) = \Theta(n) = \Theta\left(n^{\log_2 2} \log^0 n\right)$
- **Verify the hypothesis**: we are in case 2 of the Master Theorem with $k = 0$
- **Conclude**: $S(n) = \Theta\left(n^{\log_2 2} \log^{0+1} n\right) = \Theta(n \log n)$

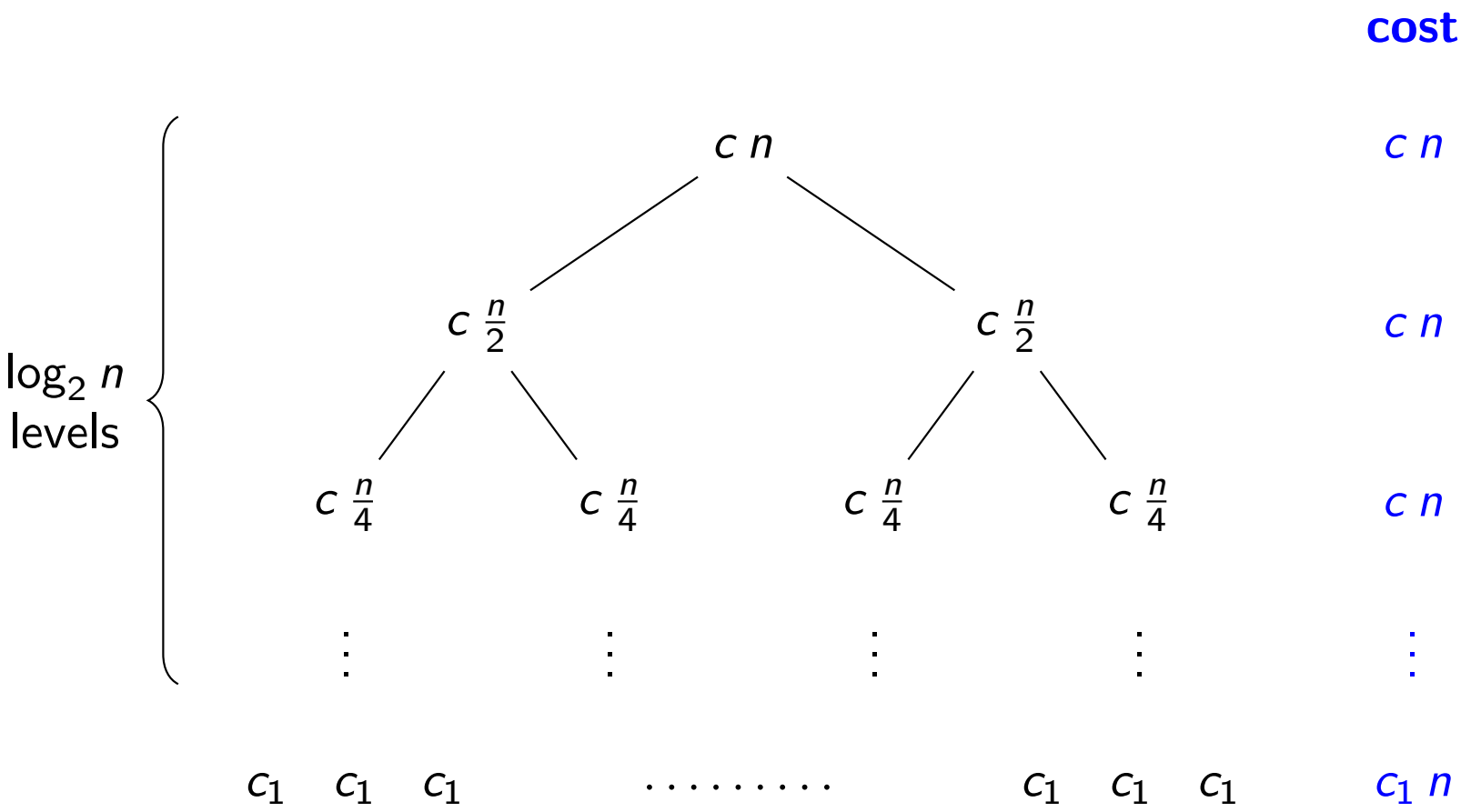
Solution by the recursion tree

- Let $n = 2^k$ be a power of 2, and let $c\,n$ be the cost of the non-recursive part of a call on $n \geq 2$ elements
- In the recursion tree, the level h (for $h = 0, 1, \ldots, k-1$) contains 2^h calls on $n/2^h$ elements each, with total cost

$$2^h \cdot c\,\frac{n}{2^h} = c\,n.$$

- The last level ($h = k = \log_2 n$) contains the n leaves, with total cost $c_1 n$
- The tree is in the next slide

The recursion tree of `FindMaximumSubarray`



Summing over the levels

- Summing over all the levels:

$$S(n) = \underbrace{c\,n + c\,n + \cdots + c\,n}_{\log_2 n \text{ times}} + c_1 n$$
$$= c\,n \log_2 n + c_1 n = \Theta(n \log n).$$

- The recursion tree also explains the result: there are $\log_2 n$ levels and each level costs $\Theta(n)$, because the crossing computations of the calls of a level scan, all together, the whole array once

- Under the hypothesis that every entry uses a constant number of bits, $|I| = \Theta(n)$, and also $\log n = \Theta(\log |I|)$ (as shown for the Recursive Binary Search). Hence

$$S(n) = \Theta(n \log n) = \Theta(|I| \log |I|).$$

Worst-case running time of `FindMaximumSubarray`

If all input values use a fixed constant number of bits,
then the worst-case running time
of the `FindMaximumSubarray` algorithm is

$$T(|I|) = \Theta(|I| \log |I|).$$

Thus, the divide and conquer algorithm is asymptotically faster
than the brute force algorithm,
whose worst-case running time is $\Theta(|I|^2)$.

Remark: an iterative linear algorithm

- The divide and conquer algorithm is much better than the brute force algorithm
- However, the maximum subarray problem can be solved even faster, with a simple **iterative** algorithm due to J. Kadane, which scans the array only **once**
- **Idea**: for each position i, let $M(i)$ be the maximum sum of a subarray **ending at** position i
- A subarray ending at i is either $a[i]$ alone, or a subarray ending at $i-1$ extended with $a[i]$. Therefore,

$$M(1) = a[1], \qquad M(i) = a[i] + \max\{0, M(i-1)\} \quad (i \geq 2).$$

Restart or extend

- In words: if the best subarray ending at $i-1$ has a negative sum, it is better to **restart** from $a[i]$; otherwise, it is better to **extend** it
- The answer to the problem is the maximum of $M(1), \ldots, M(n)$
- In the next slides, the pseudo-code of the `Kadane` algorithm. The variable *cur* stores $M(i)$ and *cI* the first index of the corresponding subarray

Algorithm: Kadane (1/2)

Input : An array $a[1..n]$ of n integer numbers
Output : Indices $bl \leq br$ of a maximum subarray and its sum *best*

```
1  function Kadane(a, n)
2      best := a[1]; bl := 1; br := 1
3      cur := a[1]; cl := 1
4      i := 2
5      while i ≤ n do
6          if cur < 0 then
7              cur ← a[i]; cl ← i                    // restart
8          else
9              cur ← cur + a[i]                      // extend
10         end
```

Algorithm: Kadane (2/2)

```
function Kadane(a, n) (continued)
    while i ≤ n (continued) do
11      if cur > best then
12          best ← cur; bl ← cl; br ← i
13      end
14      i ← i + 1
15  end
16  return (bl, br, best)
17 end
```

Run in Colab: Python · C++ · Java

Lemma 2: loop invariant of Kadane

At the beginning of each iteration of the `while` *loop (and when the loop terminates), with current index* i*:*

1. $cur = \sum_{k=cl}^{i-1} a[k]$ *is the maximum sum of a subarray* ***ending at*** *position* $i - 1$*;*
2. $best = \sum_{k=bl}^{br} a[k]$ *is the maximum sum of a subarray of* $a[1..i-1]$*.*

Proof (1/3)

Initialization ($i = 2$). The only subarray ending at 1 is $a[1..1]$, and it is also the only subarray of $a[1..1]$: $cur = best = a[1]$ with $cl = bl = br = 1$.

Maintenance. Assume the invariant holds at the beginning of the iteration with index i.

- A subarray ending at i is either $a[i..i]$, with sum $a[i]$, or $a[k..i]$ with $k < i$, whose sum is

$$\sum_{h=k}^{i-1} a[h] + a[i] \leq cur + a[i]$$

(by part 1), with equality for $k = cl$

Proof (2/3)

- Hence the maximum sum of a subarray ending at i is

$$\max\{a[i], cur + a[i]\},$$

  which is $a[i]$ if $cur < 0$ and $cur + a[i]$ otherwise: this is exactly what lines 6–9 compute. So part 1 holds for $i + 1$

- A subarray of $a[1..i]$ either is a subarray of $a[1..i-1]$ (best sum: *best*, by part 2) or ends at i (best sum: the new *cur*). Lines 11–12 keep the larger of the two, so part 2 holds for $i + 1$

Proof (3/3)

Termination. The loop terminates with $i = n + 1$: by part 2, *best* is the maximum sum of a subarray of $a[1..n]$, attained by $a[bl..br]$. □

Running time.

- The `while` loop executes $n - 1$ iterations, each with a constant number of steps
- Hence $S(n) = \Theta(n)$ and, if every entry uses a constant number of bits, $T(|I|) = \Theta(|I|)$
- Since any algorithm must read all the entries of the array, no algorithm can be asymptotically faster

Example (1/2)

Execution of `Kadane` on the stock example $a = [13, -3, -25, 20, -3, -16, -23, 18, 20, -7, 12, -5, -22, 15, -4, 7]$. Each row reports the values at the end of the iteration with index i (row $i = 1$: initialization).

i	$a[i]$	action	cur	cl	$best$	(bl, br)
1	13	initialization	13	1	13	$(1, 1)$
2	-3	extend	10	1	13	$(1, 1)$
3	-25	extend	-15	1	13	$(1, 1)$
4	20	restart ($cur < 0$)	20	4	**20**	$(4, 4)$
5	-3	extend	17	4	20	$(4, 4)$
6	-16	extend	1	4	20	$(4, 4)$
7	-23	extend	-22	4	20	$(4, 4)$
8	18	restart ($cur < 0$)	18	8	20	$(4, 4)$

Example (2/2)

i	$a[i]$	action	cur	cl	$best$	(bl, br)
9	20	extend	38	8	**38**	$(8, 9)$
10	-7	extend	31	8	38	$(8, 9)$
11	12	extend	43	8	**43**	$(8, 11)$
12	-5	extend	38	8	43	$(8, 11)$
13	-22	extend	16	8	43	$(8, 11)$
14	15	extend	31	8	43	$(8, 11)$
15	-4	extend	27	8	43	$(8, 11)$
16	7	extend	34	8	43	$(8, 11)$

The algorithm returns $(8, 11, 43)$, the same maximum subarray found by the divide and conquer algorithm, after a single scan of the array.

Summary

Key points on the maximum subarray problem:

- brute force algorithm: $\Theta(n^2)$
- divide and conquer algorithm (recursive): $\Theta(n \log n)$, with the same recurrence as Merge Sort
- Kadane algorithm (iterative): $\Theta(n)$

The lesson:

- A recursive divide and conquer algorithm is not necessarily the best possible one: here, a simple iterative algorithm that exploits the structure of the problem ($M(i)$ depends only on $M(i-1)$) is asymptotically faster
- We will come back to this lesson in the chapter «Recursive versus iterative algorithms»

Summary of the chapter

Introduction: Summary of the chapter

The four problems of the chapter cover the three typical shapes of a recursive algorithm on arrays:

- two calls on the halves with **constant** combination work (sum)
- two calls on the halves with **linear** combination work (Merge Sort, maximum subarray)
- **one** call on a half (binary search)

The recursive algorithms of the chapter (1/2)

The recurrences:

recursive algorithm	recurrence
`RecursiveSum`	$S(n) = 2S(n/2) + c$
`RecursiveSumLinear`	$S(n) = S(n-1) + 3$
`MergeSort`	$S(n) = 2S(n/2) + \Theta(n)$
`RecursiveBinarySearch`	$S(n) = S(\lfloor n/2 \rfloor) + 5$
`FindMaximumSubarray`	$S(n) = 2S(n/2) + \Theta(n)$

The recursive algorithms of the chapter (2/2)

Number of steps and extra memory:

recursive algorithm	number of steps	extra memory
`RecursiveSum`	$7n - 5 = \Theta(n)$	$\Theta(\log_2 n)$
`RecursiveSumLinear`	$3n - 1 = \Theta(n)$	$\Theta(n)$
`MergeSort`	$\Theta(n \log_2 n)$	$\Theta(n)$
`RecursiveBinarySearch`	$5(\lfloor \log_2 n \rfloor + 1) + 2 = \Theta(\log_2 n)$	$\Theta(\log_2 n)$
`FindMaximumSubarray`	$\Theta(n \log_2 n)$	$\Theta(\log_2 n)$

Summary (1/2)

Correctness:

- always by strong induction on the size of the sub-problem
- the non-recursive parts (`Merge`, `FindMaxCrossingSubarray`, `Kadane`) by loop invariants

Number of steps:

- a recurrence, solved with the recursion tree, by expansion, or with the Master Theorem
- case 1 for the sum (the leaves dominate), case 2 for Merge Sort, the maximum subarray and the binary search (all the levels have the same cost)

Summary (2/2)

When does recursion pay off?

- Recursion gives an asymptotic gain only when the work of the combine step, summed over a level, is small compared with what an iterative algorithm does (Merge Sort against the $\Theta(n^2)$ sorting algorithms)
- For the sum and the search it gives no gain in time and costs memory
- For the maximum subarray the best algorithm is iterative

Memory:

- the call stack costs memory proportional to the recursion depth